

Exception Handling in BusBookingSystem

Source code GitHub Url - [https://github.com/Padmavathi-SJ/Bus Ticket Booking](https://github.com/Padmavathi-SJ/Bus_Ticket_Booking)

→ so far I was handling the exceptions and errors using try-catch blocks with console.WriteLine statements for debugging.

→ now I have created a '**GlobalExceptionHandler**' to handle the exceptions globally formatted exceptions msgs, status codes, and exact error msgs.

BusBooking.API/Filters/GlobalExceptionHandler.cs:

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.AspNetCore.Mvc.Filters;
```

```
using Microsoft.EntityFrameworkCore;
```

```
namespace BusBooking.API.Filters
```

```
{
```

```
    public class GlobalExceptionHandler : IExceptionHandler
```

```
    {
```

```
        private readonly ILogger<GlobalExceptionHandler> _logger;
```

```
        public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger)
```

```
        {
```

```
            _logger = logger;
```

```
}
```

```
public void OnException(ExceptionContext context)
```

```
{
```

```
    var exception = context.Exception;
```

```
    var actionName = context.ActionDescriptor.DisplayName;
```

```
    _logger.LogError(exception, "Exception in {ActionName} at {Time}",  
actionName, DateTime.UtcNow);
```

```
    var response = new
```

```
{
```

```
        success = false,
```

```
        timestamp = DateTime.UtcNow,
```

```
        message = GetErrorMessage(exception),
```

```
        errorType = exception.GetType().Name
```

```
};
```

```
context.Result = new ObjectResult(response)
```

```
{
```

```
    StatusCode = GetStatusCode(exception)
```

```
};
```

```
context.ExceptionHandled = true;
```

```
}
```

```

private string GetErrorMessage(Exception ex)
{
    return ex switch
    {
        ArgumentException => ex.Message,
        ArgumentNullException => ex.Message,
        InvalidOperationException => ex.Message,
        UnauthorizedAccessException => ex.Message,
        KeyNotFoundException => "The requested resource was not found.",
        DbUpdateConcurrencyException => "Data was modified by another
user. Please refresh and try again.",
        DbUpdateException => "A database error occurred. Please try again.",
        _ => "An unexpected error occurred. Please try again later."
    };
}

```

```

private int GetStatusCode(Exception ex)
{
    return ex switch
    {
        ArgumentException => 400,
        ArgumentNullException => 400,
        InvalidOperationException => 409,
        UnauthorizedAccessException => 401,
    };
}

```

```

        KeyNotFoundException => 404,
        DbUpdateException => 500,
        _ => 500
    };
}
}
}

```

→ what **GlobalExceptionHandler** do:

- Centralizes all exception handling – one place to manage all errors
- Automatically caches exceptions – no need for try-catch in controllers
- Map exceptions to HTTP status codes:
 - `ArgumentException` -> 400 bad request
 - `UnauthorizedAccessException` -> 401 unauthorized
 - `KeyNotFoundException` -> 404 Not Found
 - `InvalidOperationException` -> 409 Conflict
 - Others -> 500 internal server error
- Provides consistent error response format:

```

{
  "success": false,
  "timestamp": "2024-01-01T10:00:00Z",
  "message": "Error description",
  "errorType": "ArgumentException"
}

```

- Logs all exceptions automatically – no more `Console.WriteLine`

AuthController.cs: - Before

```
[HttpPost("user-login")]

public async Task<ActionResult<AuthResponseDto>> UserLogin([FromBody]
LoginCommand command)

{
    command.Role = UserRole.Customer;

    return await ProcessLogin(command); // Complex method
}


private async Task<ActionResult<AuthResponseDto>>
ProcessLogin(LoginCommand command)

{
    try
    {
        var response = await _mediator.Send(command);

        return Ok(response);
    }
    catch (UnauthorizedAccessException ex)
    {
        return Unauthorized(new { message = ex.Message });
    }
}


[HttpPost("user-register")]
```

```

public async Task<ActionResult> RegisterCustomer([FromBody]
RegisterCustomerCommand command)
{
    try
    {
        await _mediator.Send(command);
        return Ok(new { message = "Registration successful" });
    }
    catch (Exception ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

```

AuthController.cs - After:

```

[HttpPost("user-login")]
public async Task<ActionResult<AuthResponseDto>> UserLogin([FromBody]
LoginCommand command)
{
    command.Role = UserRole.Customer;
    var response = await _mediator.Send(command); // Direct call
    return Ok(response);
}

[HttpPost("user-register")]

```

```

public async Task<ActionResult> RegisterCustomer([FromBody]
RegisterCustomerCommand command)
{
    await _mediator.Send(command);
    return Ok(new { message = "Registration successful" });
}

```

Benefits for AuthController:

Before:

- Had separate ProcessLogin helper method
- Duplicate error handling in each method
- Inconsistent response formats
- Manual unauthorizd handling
- Console output need for debugging

After:

- Removed unnecessary helper method
- No error handling code
- Consistent fromat from global filter
- Automatic 401 response
- Automatic logging

BookingController.cs: - Before

```
[HttpGet("booked-seats/trip/{tripId}")]
```

```

public async Task<ActionResult<List<string>>> GetBookedSeatsForTrip(Guid
tripId)

```

```
{
```

```

try
{
    Console.WriteLine($"[DEBUG-BOOKED-SEATS] Getting booked seats for
trip: {tripId}");

    var result = await _mediator.Send(new
GetBookedSeatsForTripQuery(tripId));

    Console.WriteLine($"[DEBUG-BOOKED-SEATS] Found {result.Count}
booked seats");

    return Ok(result);
}
catch (Exception ex)
{
    Console.WriteLine($"[ERROR-BOOKED-SEATS] Failed: {ex.Message}");

    return BadRequest(new { message = ex.Message });
}
}

```

[HttpPost]

[Authorize]

```

public async Task<ActionResult<BookingConfirmationDto>>
CreateBooking([FromBody] CreateBookingDto dto)

```

```

{
    try
    {
        var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;

```

```

        if (string.IsNullOrEmpty(userId))
        {
            return Unauthorized(new { message = "User not authenticated" });
        }
        // ... more code
    }
    catch (Exception ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

```

BookingController.cs - After:

```

[HttpGet("booked-seats/trip/{tripId}")]
public async Task<ActionResult<List<string>>> GetBookedSeatsForTrip(Guid tripId)
{
    var result = await _mediator.Send(new GetBookedSeatsForTripQuery(tripId));
    return Ok(result);
}

```

```

[HttpPost]

```

```

[Authorize]

```

```

public async Task<ActionResult<BookingConfirmationDto>>
CreateBooking([FromBody] CreateBookingDto dto)

```

```
{  
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;  
    if (string.IsNullOrEmpty(userId))  
    {  
        throw new UnauthorizedAccessException("User not authenticated");  
    }  
    // ... rest of code, no try-catch  
    var result = await _mediator.Send(command);  
    return Ok(result);  
}
```

Benefits for BookingController:

Before:

- 3 try-catch blocks
- Console.WriteLine debugging scattered
- Manual unauthoried returns
- Inconsistent error formats
- ~50 lines of error handling code

After:

- 0 try-catch blocks
- Automatic structured logging
- Throws exception (global filter handles)
- Consistent format
- ~0 lines of error handling code

Busescontroller.cs – Before:

```
[HttpGet("search-trips")]
```

```
public async Task<ActionResult> SearchTrips(
```

```
    [FromQuery] string? source,
```

```
    [FromQuery] string? destination,
```

```
    [FromQuery] string? tripDate)
```

```
{
```

```
    try
```

```
    {
```

```
        DateTime? parsedDate = null;
```

```
        // ... parsing logic
```

```
        if (!string.IsNullOrEmpty(tripDate) && !DateTime.TryParse(tripDate, out var  
date))
```

```
        {
```

```
            return BadRequest(new { message = "Invalid date format. Use YYYY-  
MM-DD" });
```

```
        }
```

```
        var result = await _mediator.Send(new SearchTripsQuery(source, destination,  
parsedDate));
```

```
        return Ok(result);
```

```
    }
```

```
    catch (Exception ex)
```

```
    {
```

```
        Console.WriteLine($"[ERROR-TRIPS] Search failed: {ex.Message}");
```

```

        return BadRequest(new { message = ex.Message });
    }
}

```

```

[HttpGet("trips/{tripId}")]
public async Task<ActionResult> GetTripById(Guid tripId)
{
    try
    {
        var result = await _mediator.Send(new GetTripByIdQuery(tripId));
        return Ok(result);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[ERROR-TRIP] Get trip failed: {ex.Message}");
        return NotFound(new { message = ex.Message });
    }
}

```

Busescontroller.cs – After:

```

[HttpGet("search-trips")]
public async Task<ActionResult> SearchTrips(
    [FromQuery] string? source,

```

```

[FromQuery] string? destination,
[FromQuery] string? tripDate)
{
    DateTime? parsedDate = null;

    if (!string.IsNullOrEmpty(tripDate))
    {
        if (!DateTime.TryParse(tripDate, out var date))
        {
            throw new ArgumentException("Invalid date format. Use YYYY-MM-DD");
        }
        parsedDate = DateTime.SpecifyKind(date.Date, DateTimeKind.Utc);
    }

    var result = await _mediator.Send(new SearchTripsQuery(source, destination,
parsedDate));
    return Ok(result);
}

[HttpGet("trips/{tripId}")]
public async Task<ActionResult> GetTripById(Guid tripId)
{
    var result = await _mediator.Send(new GetTripByIdQuery(tripId));
    return Ok(result);
}

```

```
}
```

Benefits for Busescontroller:

Before:

- Try-catch in every endpoint
- Manual BadRequest returns
- Manual NotFound returns
- Console output in catch blocks
- Inconsistent error responses

After:

- No try-catch needed
- Throws argumentException
- Global filter handles 404
- Automatic logging
- Uniform format

OperatorController.cs - Before

```
[HttpPost("schedule-trip")]
```

```
public async Task<ActionResult> ScheduleTrip([FromBody]  
ScheduleTripCommand command)
```

```
{
```

```
    try
```

```
    {
```

```

var userId = GetUserId();
// ... validation logic
if (busOperator == null)
{
    return Unauthorized(new { message = "Operator profile not found" });
}
var tripId = await _mediator.Send(command);
return Ok(new { id = tripId, message = "Trip scheduled successfully!" });
}
catch (Exception ex)
{
    Console.WriteLine($"[ERROR] Failed to schedule trip: {ex.Message}");
    return BadRequest(new { message = ex.Message });
}
}

```

[HttpGet("notifications")]

```

public async Task<ActionResult> GetNotifications([FromQuery] bool unreadOnly
= false)

```

```

{
    try
    {
        var userId = GetUserId();
        var result = await _mediator.Send(...);
        return Ok(result);
    }
}

```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine($"[ERROR] Failed to fetch notifications: {ex.Message}");
        return BadRequest(new { message = ex.Message });
    }
}

```

OperatorController.cs - After:

```

[HttpPost("schedule-trip")]
public async Task<ActionResult> ScheduleTrip([FromBody]
ScheduleTripCommand command)
{
    var userId = GetUserId();

    var busOperator = await _context.BusOperators
        .FirstOrDefaultAsync(o => o.UserId == userId);

    if (busOperator == null)
    {
        throw new UnauthorizedAccessException("Operator profile not found");
    }

    var bus = await _context.Buses

```

```
.FirstOrDefaultAsync(b => b.Id == command.BusId && b.OperatorId ==  
busOperator.Id);
```

```
if (bus == null)  
{  
    throw new UnauthorizedAccessException("Bus not found or does not belong  
to you");  
}
```

```
var tripId = await _mediator.Send(command);  
return Ok(new { id = tripId, message = "Trip scheduled successfully!" });  
}
```

```
[HttpGet("notifications")]
```

```
public async Task<ActionResult> GetNotifications([FromQuery] bool unreadOnly  
= false)
```

```
{  
    var userId = GetUserId();  
  
    var result = await _mediator.Send(new GetOperatorNotificationsQuery(userId,  
unreadOnly));  
  
    return Ok(result);  
}
```

Benefits for OperatorController:

Before:

- Every endpoint wrapped in try-catch
- Manual unauthorized returns
- Console.writeline scattered
- Multiple error response formats
- ~100 lines of error handling

After:

- No try-catch blocks
- Throws UnauthorizedAccessException
- Structured logging
- Consistent format
- ~0 lines

- ➔ Controllers now focus only on routing and orchestration – no error handling logic
- ➔ Exceptions are thrown, not caught – let them bubble up to global handler
- ➔ Global filter handles all exceptions uniformly – one place to rule them all
- ➔ Proper http status codes – 400, 401, 404, 409, 500 based on exception type
- ➔ Automatic logging – every exception is logged with context
- ➔ Consistent client responses – frontend always receives same error format.